

Permisos, seguridad y guardrails

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 12 de 12 · Notas en español (es-ES)

La historia inquietante era simple: Claude podría emitir `rm -rf /` (o `/root`) y Bash podría obedecer. Las herramientas integradas son potentes; las **compuertas de permiso** y los **hooks** son cómo un agente de empresa se queda en el lado correcto del disco. El formador contó **seis compuertas**, **cinco modos de permiso** y un control crítico de «no bucles infinitos» — luego mostró combinaciones en un envoltorio `run_agent`.

Seis compuertas (lista de aula)

Compuerta	Qué hace	Uso típico
Hooks previos al uso de herramienta	Inspeccionar la llamada inminente; parar/denegar con motivo	Bloquear patrones bash peligrosos antes de que corran
Herramientas no permitidas	Lista de denegación dura: nunca ejecutar, aunque el modelo lo pida	Sin Write/Edit en un agente revisor de solo lectura
Herramientas permitidas	Lista de herramientas integradas, a medida o MCP	Solo Read, Glob, Grep para observar
Modos de permiso	Si hay que preguntar a un humano, autoaprobar o denegar por defecto	Ved los cinco modos abajo
<code>can_use_tool</code>	Retrollamada cuando hace falta recolección interactiva	Preguntar al usuario; reunir datos extra
Sin retrollamada / no interactivo	Sesiones sin humano; evitar el ping-pong infinito de herramientas	Trabajos por lotes autenticados

Lo denegado gana. Si Bash está permitido pero `rm *` también está denegado (o un hook lo niega), prima la denegación. Las herramientas permitidas significan «elegibles», no «ya aprobadas para cualquier argumento».

COMPROBAD LOS NOMBRES VIGENTES DEL ENUMERADO

Las diapositivas listaban `default`, `acceptEdits`, `bypassPermissions`, `don't ask`, `plan` y un «auto» que también conocía el linter. Emparejad las cadenas con la documentación vigente de permisos del Agent SDK / Claude Code, no con una captura del 27 de agosto de 2026.

Cinco modos que quería que contrastarais

- **Default** — comportamiento estándar; pregunta en operaciones peligrosas; las herramientas no emparejadas pueden ir a `can_use_tool`. El estado interactivo más general.
- **Don't ask** — sin UI de permiso; lo que no esté preaprobado en la lista se *deniega*, pero el agente *sigue* con pasos posteriores (falla el write, un Read posterior aún puede correr).
- **Accept edits** — autoaprobar ediciones de sistema de ficheros que ya están en la lista.

- **Bypass permissions** — tratar las herramientas listadas como aprobadas en la compuerta. Lo llamó peligroso; solo con una lista estrecha.
- **Plan** — planificación de solo lectura; sin ediciones de fichero o sistema. Usadlo al inspeccionar; pasad a accept-edits cuando pretendáis escribir.

Ejemplos de espectro

Solo observar: permitir Read, Glob, Grep; modo don't-ask. Aunque el modelo emita Write, no aterriza.

Lectura-escritura en un conjunto conocido: permitir Read/Write/Edit (y quizá Bash); don't-ask o accept-edits. Las herramientas MCP que no estén en la lista se quedan fuera en don't-ask. **Planificar y luego**

actuar: agente en modo plan para redactar; segundo agente con accept-edits para aplicar.

Qué mostró el cuaderno

Prompt: leer `auth.py` y guardar un resumen de una frase en una carpeta de demo. Sin Write permitido y con don't-ask, el resultado fue: se intentó escribir, permiso denegado, ficheros igual, a veces una oferta de habilitar Write. El mismo prompt con Write permitido creó `summary_b.txt`. Una ejecución posterior mostró don't-ask saltándose un Write/Bash bloqueado y ejecutando aún un Read siguiente.

Bloqueo quirúrgico con hooks previos

Permitir Bash por completo es tosco. Los hooks (pre-tool-use, post-tool-use, fallo, envío de prompt, stop, inicio/fin de subagente, pre-compact, petición de permiso, notificación) dejan emparejar patrones. El repositorio listaba patrones bash peligrosos: si la herramienta es Bash y el comando coincide, devolver denegación más motivo (`continue = false` en el matcher). Eso detuvo `rm -rf` en una carpeta `blocking-demo` aunque Bash estuviera permitido.

Segundo patrón: **redirigir escrituras** lejos de una ruta protegida hacia un directorio temporal para no tirar tokens facturados. La salida del LLM es cara; si Write se deniega en `/protected`, el hook reescribe a una carpeta segura y lo registra.

Guardrails como validación de entrada/salida

Los permisos detienen herramientas. Los **guardrails** en esta hora eran sobre todo comprobaciones del desarrollador alrededor del prompt y de la salida del modelo/herramienta: bloquear «ignora las instrucciones previas» / «revela el prompt de sistema», limitar longitud, buscar secretos, redactar. Aplicad *antes de query* (ahorráis tokens) y *después* (no registréis claves API que el modelo haya repetido). Vijay coincidió en que es ingeniería general, no un limitador oculto de tokens de Claude. Señaló listas de código abierto (Rulebricks, repos de guardrails al estilo Claude Code) en lugar de un único import oficial. La guía oficial de evaluación está en la página de `strengthen-guardrails` de Anthropic.

El ejemplo de PII que no es «cifrar en la pasarela»

Rishabh preguntó si HIPAA/PII significaba cifrado tipo Protegrity hacia la pasarela. La respuesta de agente de Vijay: un agente de BD de clientes que arma un informe y envía correo *debe* ver una dirección real para enviar, pero los registros, los saltos MCP y las herramientas de terceros no deben filtrar esa dirección. Permisos, hooks y escáneres de salida son la capa de la app Claude; no sustituyen la arquitectura de cifrado de vuestra organización.

Documentación oficial

- **Reforzar guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Guía de evaluación y guardrails de Anthropic.

- **Política de uso**

<https://www.anthropic.com/legal/usage-policy>

Lo que no podéis construir, haya hooks o no.

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Modos de permiso en el producto que ya usasteis en las semanas 2–3.

- **Visión general de tool use**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Las herramientas a medida siguen pasando por las mismas compuertas si el SDK las envuelve.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.